

Phase 2 — Sequence Generation Specification

LOCKED DESIGN DOCUMENT — Review Before Writing Any GRU Code

Status:  LOCKED — Do not modify without a formal review and version increment

Version: v1.0

Approved by: Phase 1.5E review (2026-09-02)

Prerequisite: Phase 1.5 complete; E1 LightGBM baseline = PR-AUC 0.5317

0. Research Question

"Does historical transaction behavior provide statistically and operationally significant predictive information beyond conventional tabular fraud features?"

Phase 2 answers this question by training a GRU sequence model (E2) and comparing it against the frozen E1 baseline using bootstrap confidence intervals and operational metrics. **All outcomes (GRU better, equal, or worse) are valid research results.**

1. Entity Definition

Parameter	Value	Justification
Entity key	card1	Most stable card-level identifier in IEEE-CIS. Has 13,553 unique entities with a wide transaction-count distribution.
Why not card1+card2+card3+card6?	Composite key creates too many singleton entities	card1 alone provides sufficient sequence depth for 48.05% of entities
Why not client_id?	Not available in IEEE-CIS	card1 is the closest proxy for a consistent card entity

Entity distribution (from Phase 1 forensic analysis):

Total card1 entities:	13,553
Entities with ≥ 5 transactions:	6,518 (48.05%)
Median transactions per entity:	4
Max transactions per entity:	~ 3,000+ (heavy users)

2. Minimum Sequence Length

Parameter	Value	Reasoning
Minimum history length	5	At least 4 historical transactions needed to form one prediction window; entities with < 5 are excluded from GRU training
Effect	~51.95% of card1 entities excluded from GRU	These transactions still appear in E1 LightGBM comparison — they receive tabular predictions only
Maximum sequence length	20 (capped for GRU)	Beyond 20, GRU gradient flow degrades; longer histories padded/truncated

[!IMPORTANT] Entities excluded from GRU training are NOT excluded from the evaluation comparison. The test set for both E1 and E2 is identical: all test-split transactions. For entities with < 5 history transactions, E2 must handle gracefully (use LightGBM prediction only in fusion, or apply GRU with shorter sequences if possible).

3. Temporal Ordering Rule

All transactions within a sequence MUST be ordered by **strictly ascending TransactionDT**.

Card A entity – CORRECT order:
TransactionDT: 86,400 → 173,200 → 259,800 → 432,000 → 604,700
 T1 T2 T3 T4 T5

No shuffling is ever permitted within a sequence.
If two transactions share identical TransactionDT (tie), use TransactionID as a stable secondary sort key (ascending).

4. Sliding Window Construction

Each entity's transaction history is converted into overlapping windows of length L (window_size), predicting the label of the **last** transaction in each window.

4.1 Window Definition

For a card entity with history [T1, T2, T3, T4, T5, T6, T7]:

Window 1: inputs = [T1, T2, T3, T4] → label = T5.isFraud
Window 2: inputs = [T2, T3, T4, T5] → label = T6.isFraud
Window 3: inputs = [T3, T4, T5, T6] → label = T7.isFraud

- Formal definition:**
- Window size L = 5 (4 history steps + 1 prediction target)

- For each entity with $N \geq L$ transactions ordered $T_1 < T_2 < \dots < T_n$:
 - For i from L to N : $\text{window}_i = \text{inputs} = [T_{i-L+1}, \dots, T_{i-1}]$, $\text{label} = T_i$
- Each window produces exactly one (sequence, label) pair
- The **label transaction (T_i) is NEVER used as input in its own window**

4.2 Feature Assignment per Timestep

At each position in the sequence (each historical T_i), use only features computable from information available at T_i 's **TransactionDT**:

- Tabular features from the processed parquet (the same feature set as E1)
- Do NOT include **isFraud** of any historical transaction as a sequence feature (this would be future target leakage)
- The label is taken only from the prediction transaction (last window element)

5. Causal History Rule (CRITICAL — replaces strict same-split boundary)

[!IMPORTANT] This is the most significant design decision in this specification. Read carefully.

5.1 The Problem with Strict Same-Split Boundary

A naive approach would restrict sequence histories to only transactions within the same split:

```
Train window:      only uses train-split transactions as history
Validation window: only uses validation-split transactions as history
Test window:       only uses test-split transactions as history
```

This is **too conservative** and unrealistic. It means: when predicting a validation transaction on Day 130, the model pretends transactions from Days 1–127 (the training split) never happened. But in real deployment, those historical transactions ARE known.

5.2 Correct Causal History Rule

The rule is: **a window may use any transaction as history as long as that transaction's TransactionDT is strictly less than the prediction transaction's TransactionDT.**

The split label of the prediction (label transaction) determines which evaluation split the window belongs to. The history transactions may come from any earlier split.

TRAIN WINDOWS

```
Prediction label  $T_i \in$  train split ( $\text{TransactionDT} < \text{split\_val\_dt}$ )
History  $T_{i-L+1} \dots T_{i-1}$ : all have  $\text{TransactionDT} < T_i.\text{TransactionDT}$  ✓
(All history is necessarily in train split – no cross-split issue here)
```

VALIDATION WINDOWS

```
Prediction label  $T_i \in$  validation split ( $\text{split\_val\_dt} \leq \text{TransactionDT} < \text{split\_test\_dt}$ )
History  $T_{i-L+1} \dots T_{i-1}$ : may include train-split transactions ✓
```

Rationale: when predicting a transaction on Day 132, history from Days 1–131 is known and should be used. This mirrors real deployment.

TEST WINDOWS

Prediction label $T_i \in \text{test split}$ ($\text{TransactionDT} \geq \text{split_test_dt}$)

History $T_{\{i-L+1\}} \dots T_{\{i-1\}}$: may include train and validation split transactions



Rationale: same deployment logic – when predicting Day 160, all prior transactions on that card (Days 1–159) are known.

5.3 What Remains Forbidden

Even with causal history, the following are ALWAYS forbidden:

Rule	Why
T_i may not appear in its own input sequence	The model cannot see the current transaction before predicting it
Any transaction with $\text{TransactionDT} > T_i.\text{TransactionDT}$ in history	Future transactions are unknown
isFraud of ANY historical transaction as a sequence feature	Using past fraud labels as features would be target leakage if used to learn patterns
GRU model fitted on validation or test data	Model trained on train windows only

5.4 Label Accessibility Summary

Prediction transaction split	Labels used for GRU training?	Labels used for evaluation?
Train	✓ Yes (supervision signal)	✓ Yes (training metrics only)
Validation	✗ No	✓ Yes (validation metrics)
Test	✗ No	✓ Yes (test metrics — evaluated once)

6. Split Assignment for Windows

Window split = split of the prediction transaction (label T_i)

```
split_val_dt = 10,972,800    (Day 127.0, from Phase 1 split_meta.json)
split_test_dt = 13,392,000   (Day 155.0, from Phase 1 split_meta.json)
```

```
if T_i.TransactionDT < split_val_dt: window → train
elif T_i.TransactionDT < split_test_dt: window → validation
else:                                window → test
```

7. Required Automated Leakage Tests for Phase 2

Before training any GRU, the window-generation code must pass **all 5 of the following tests**:

Test #	Name	Check
T1	<code>no_self_input</code>	For every window, the prediction TransactionID does not appear in the input sequence TransactionIDs
T2	<code>no_future_in_history</code>	For every window, all input TransactionDTs are strictly less than the prediction TransactionDT
T3	<code>no_label_as_feature</code>	<code>isFraud</code> column is not included in any sequence feature matrix
T4	<code>temporal_ordering</code>	All input sequences are ordered by ascending TransactionDT within each window
T5	<code>train_model_only_on_train_labels</code>	GRU model is never exposed to validation or test split labels during training

These tests must be implemented in `tests/test_phase2_sequences.py` before any GRU training begins.

8. Sequence Feature Set

Each timestep in the GRU sequence uses the **same feature set as E1** (406 features), minus:

- `TransactionID` (identifier, not a feature)
- `isFraud` (target — never used as sequence feature)
- `TransactionDT` (raw temporal counter — use `hour_sin/hour_cos` from engineered features instead)

This gives **404 features per timestep**.

Additionally, the **time gap** between consecutive transactions is a strong behavioral signal for fraud detection:

```
# Engineered per-window feature: time gap in seconds between consecutive transactions
dt_gap = [T_{i+1}.TransactionDT - T_i.TransactionDT for i in 0..L-2]
```

This adds `L-1` time-gap features as an additional input channel to the GRU.

9. GRU Architecture Sketch (to be finalized in Phase 2 code)

```
Input: [batch, L-1, 404 + 1 gap_feature]
      └─ L-1 history steps, each with 404 tabular features + 1 time gap

GRU:   hidden_size=128, num_layers=2, dropout=0.3
```

```

└─ Output: last hidden state  $h \in \mathbb{R}^{128}$ 

MLP:    128 → 64 → 1 (sigmoid)
        └─ Fraud probability from sequence alone

Fusion (E3, not Phase 2):
  [GRU embedding (128)] + [LightGBM score (1)] → MLP → final probability

```

[!NOTE] The E2 GRU is a standalone model. Fusion (E3) is a separate experiment after E2 is evaluated.

10. Evaluation Protocol

Phase 2 evaluation follows the exact same protocol as Phase 1:

1. **Test set evaluated exactly once** using frozen threshold (selected on validation)
2. **Primary metric:** PR-AUC (with bootstrap 95% CI)
3. **Secondary metrics:** ROC-AUC, Recall@1%FPR, Precision@Top-1000, F1, MCC
4. **Comparison:** Bootstrap CI overlap test against E1 ($0.5317 \pm \text{CI}$)
5. **All outcomes are scientifically valid**

11. What Sequence Generation Must NOT Do

Forbidden	Why
Use <code>isFraud</code> of past transactions as a sequence feature	Target leakage
Use future transactions as history	Temporal leakage
Include the prediction transaction T_i as part of its own input	Self-reference leakage
Fit any normalisation scaler using validation or test data	Preprocessing leakage
Create windows that span a train→test boundary for the GRU model training	Model trains only on train labels; history from prior splits is allowed

12. File Structure for Phase 2

```

src/
  sequences/
    __init__.py
    window_generator.py    # Creates (X_seq, y) pairs from processed parquets
    sequence_dataset.py    # PyTorch Dataset for GRU training

  models/
    gru_fraud.py           # GRU model class

```

```
experiments/  
  run_phase2_gru.py      # Phase 2 training orchestrator  
  E2_gru/                # All Phase 2 outputs  
  
tests/  
  test_phase2_sequences.py # 5 mandatory leakage tests  
  
reports/phase2/  
  SEQUENCE_GENERATION_SPEC.md ← This document  
  phase2_gru_report.md       ← Generated after training
```

13. Frozen Parameters from Phase 1

The following values from Phase 1 must NOT be re-derived or changed for Phase 2:

Parameter	Value	Source
split_val_dt	10,972,800	experiments/E1_lightgbm/split_meta.json
split_test_dt	13,392,000	experiments/E1_lightgbm/split_meta.json
Random seed	42	configs/phase1_lightgbm.yaml
E1 frozen threshold	0.616521	experiments/E1_lightgbm/metrics.json
E1 test PR-AUC	0.531731	experiments/E1_lightgbm/metrics.json
Feature column list	406 features	experiments/E1_lightgbm/feature_names.json

Document version: v1.0 | Locked: 2026-09-02 | Phase 1.5E complete
All GRU code must comply with this specification before merging.